

```
/*  
 * P&O MPPT Algorithm - C++ Desktop  
Simulation  
 * SIT Tumakuru | Rishi R (1SI23EE046) |  
2025-26  
 * C++ simulation  
 * Run:    ./mppt_sim  
 */
```

```
#include <iostream>  
#include <iomanip>  
#include <cmath>  
#include <vector>  
using namespace std;
```

```
class PVPanel {  
    double Voc, Isc, irradiance;  
public:  
    PVPanel() : Voc(18.0), Isc(3.5),  
irradiance(1000.0) {}  
    void setIrradiance(double irr) {
```

```

    irradiance = irr;
    Isc = 3.5 * (irradiance / 1000.0);
    Voc = 18.0 * (1 + 0.01 * log(irradiance
/ 1000.0 + 0.001));
}
double getCurrent(double V) {
    if (V <= 0 || V >= Voc) return 0;
    double I = Isc * (1 - exp((V - Voc) / (0.1
* Voc)));
    return max(0.0, I);
}
double getPower(double V) { return V *
getCurrent(V); }
double getVoc() { return Voc; }
};

```

```

class MPPTController {
    double dutyCycle, prevPower, prevDuty,
deltaD, minD, maxD;
public:
    MPPTController() : dutyCycle(0.25),

```

```
prevPower(0), prevDuty(0.25),  
        deltaD(0.015), minD(0.25),  
maxD(0.85) {}  
    double update(double pvPower) {  
        double dP = pvPower - prevPower;  
        if (dP > 0)  
            dutyCycle += (dutyCycle >=  
prevDuty) ? deltaD : -deltaD;  
        else if (dP < 0)  
            dutyCycle += (dutyCycle >=  
prevDuty) ? -deltaD : deltaD;  
        dutyCycle = max(minD, min(maxD,  
dutyCycle));  
        prevPower = pvPower; prevDuty =  
dutyCycle;  
        return dutyCycle;  
    }  
    double getDutyPercent() { return  
dutyCycle * 100.0; }  
};
```

```
class Battery {
    double voltage, chargeLevel;
public:
    Battery() : voltage(11.5),
chargeLevel(0.75) {}
    void charge(double current, double
timeSec) {
        if (voltage >= 14.4) return;
        chargeLevel += (current * timeSec /
3600.0) / 7.0;
        chargeLevel = min(1.0, chargeLevel);
        voltage = 12.0 * (0.85 + 0.15 *
chargeLevel) + current * 0.1;
        voltage = min(14.5, voltage);
    }
    double getVoltage() { return voltage; }
    double getCharge() { return chargeLevel
* 100.0; }
};
```

```
int main() {
```

```
PVPanel panel;  
MPPTController mppt;  
Battery battery;
```

```
vector<pair<int,double>> profile = {  
    {1,500},{10,700},{20,1000},{35,800},  
    {45,600},{55,900}  
};
```

```
cout << string(75,'=') << endl;  
cout << "  MPPT P&O Simulation | SIT  
Tumakuru EEE | Rishi R" << endl;  
cout << string(75,'=') << endl;  
cout << left << setw(6)<<"Iter" <<  
setw(10)<<"Irrad"  
    << setw(9)<<"PV_V" <<  
setw(9)<<"PV_I"  
    << setw(9)<<"PV_P" <<  
setw(9)<<"Duty%"  
    << setw(9)<<"BatV" <<  
setw(8)<<"Bat%" << endl;
```

```
cout << string(75,'-') << endl;
```

```
int pi = 0;
```

```
double irr = 500.0;
```

```
for (int i = 1; i <= 60; i++) {  
    if (pi < (int)profile.size() && i >=  
profile[pi].first)  
        irr = profile[pi++].second;
```

```
    panel.setIrradiance(irr);  
    double V = panel.getVoc() * (1.0 -  
mppt.getDutyPercent()/100.0 * 0.4);  
    V = max(0.0, min(panel.getVoc(), V));  
    double I = panel.getCurrent(V);  
    double P = panel.getPower(V);  
    double duty = mppt.update(P);  
    battery.charge(I * 0.85, 10.0);
```

```
cout << left << setw(6) << i  
    << setw(10) << fixed <<
```

```

setprecision(0) << irr
    << setw(9) << setprecision(2) << V
    << setw(9) << I
    << setw(9) << P
    << setw(9) << setprecision(1) <<
duty*100
    << setw(9) << setprecision(2) <<
battery.getVoltage()
    << setw(8) << setprecision(1) <<
battery.getCharge()
    << endl;
}

cout << string(75,'=') << endl;
cout << "Final Battery: " <<
battery.getVoltage() << "V | "
    << battery.getCharge() << "% charged"
<< endl;
return 0;
}

```